AVC recovery & sync triage — verified findings against HEAD

`9055449`

Scope: independent verification of every defect described in `AVC_RECOVERY_AND_SYNC_ALGO_AND_BUGS.md`, run to ground against the current source tree (HEAD `9055449`, 2026-07-19).

Method: six parallel investigations, each independently and adversarially re-verified against source (every claim below carries a `file:line` citation that was re-checked by a second pass). Two numeric errors in the original document were found and corrected in the process — they are called out inline. A separately produced third-party triage report (repo-only analysis) reached the same three primary conclusions independently, which raises confidence further; one finding below (§5) is new and has no counterpart in either the document or that report.

Executive summary

The incident is **two faults chained together**, plus a routing bug that can misdirect the whole recovery machinery:

1. A **transient RX hiccup** (most likely a single errored/empty IR descriptor) is escalated into a hard timing loss by a cycle-accounting bug (§5).
2. The AV/C recovery that then fires is **structurally unable to succeed**: it restarts transport only, while the TX producer keeps its monotonic cursor, so IT prime rejects the stale cursor and streaming dies with `0xe00002c9` (§1). This is the primary bug.
3. Independently, the single timing-loss callback slot means **DICE devices never even receive their timing-loss events** — AV/C's registration overwrites DICE's (§2).
4. Two further TX-side defects (frame drop without retention §3, replay-ring horizon violation §4) explain the audible click/silence with apparent self-heal, and the `[TxPrepRange] short` signature, respectively.

The document's central diagnosis is **correct**. The "raw RX/TX SYT mismatch" theory is **not supported** and should be treated as a false lead (§6).

---

1. AVC-RECOVERY-001 — recovery dies with `0xe00002c9` (CONFIRMED, primary)

Root cause

A seam-ownership violation: the producer reset + prefill contract is audio-owned and runs **only** in the CoreAudio `StartIO` path. The AV/C timing-loss recovery restarts **transport only** and never re-runs it.

Verified chain

Step	Evidence
Normal start resets the producer	`ResetProducerForStart()` at `ASFWAudioDevice.cpp:229` (primary TX) and `:317` (secondary); `PrefillTxRingBeforeStart` at `:342`; prefill validated to `committedEnd == numSlots` at `:344–360`. Comment at `:225–228` already warns that a stale committed cursor fails IT prime.
No other caller exists	grep-confirmed: `ResetProducerForStart` / `PrefillTxRingBeforeStart` are called only from `ASFWAudioDevice.cpp` (`StartIO`).
Recovery path skips it	`AVCAudioBackend.cpp:272` `RecoverStreaming` → `AudioDuplexCoordinator::RunRecoveryStreaming` → `RunDuplexStop` + `RunDuplexStart` → `DuplexStartTransaction` → `hostTransport_.PrepareTransmit` → `IsochTransmitContext::Start`. No producer reset/prefill anywhere in this transaction.
Cursor is monotonic and survives	`committedEnd.store(packet.packetIndex + 1)` in `PublishSlot` (`ASFWAudioDriverPrivate.hpp:188`) — absolute index, ~13.6 M after minutes at ~8000 pkt/s. `ResetConsumerForArm()` (`IsochTxQueue.hpp:140–155`) deliberately resets **only** consumer/transport fields and leaves `committedEnd` untouched.
Prime rejects it	`IsochTransmitContext::Start()` reads `committedEnd` as `preFillCount` (`IsochTransmitContext.cpp:291`) and `IsochTxDmaRing::Prime` (`IsochTxDmaRing.cpp:152–161`) requires `kNumPackets ≤ preFillCount ≤ numSlots`. Rejection → `packetsAssembled != 48` → `kIOReturnInternalError` → `0xe00002c9`. The Prime failure log string matches the incident verbatim.

> **Correction to the document:** the valid Prime window is **[48, 912]**, not [48, 408]. `kTxSharedSlotPackets = 912` (`AudioTimingGeometry.hpp:161`), `kNumPackets = 48` (`IsochDmaGeometry`). The 408 figure is pre-redesign geometry. This does not change the root cause — ~13.6 M is far outside either window.

Fix design (producer-owned recovery epoch)

Ownership rule: **transport never writes `committedEnd`** (the FW-60 seam rule). Only the audio producer resets it and re-prefills; transport validates and arms.

- Factor the `StartIO` reset + prefill + validate block into one reusable `PrepareTxProducerForArm(ivars, epoch)` so cold start and recovery share a single code path (no second, divergent path).
- Drive it from recovery via a new `RecoverProducer(epoch)` OSAction targeting the audio driver queue, mirroring the existing `TxPreparationReady` conduit (`IsochTransmitContext.cpp:460–462` → `ASFWAudioDriver.iig:40`, IMPL on the audio queue at `ASFWAudioDriverZts.cpp:571`). The shared control block already carries the paired request/handled generation idiom (`refillRequestGeneration` / `refillHandledGeneration`, `IsochTxQueue.hpp:124–125, 157–164`) that the epoch handshake needs.
- Sequence: mint epoch E → quiesce transport + disconnect CMP → producer runs reset + prefill on its own queue → publishes `prefillReadyEpoch == E && committedEnd == numSlots` → only then transport `ResetConsumerForArm` + `Prime` + arm.

- On timeout or any failure: run `RunDuplexStop` to completion (CMP down) and leave the engine xrun'd. Never leave CMP connected without a primed producer (no half-running state).
- Must cover **both** primary and secondary TX rings in lockstep.
- Open decision: home for the epoch field — `AudioTransportControlBlock` (audio/session concept, no transport ABI bump) vs. also mirroring it in `IsochTxQueueControl` (bump `kTxQueueAbiVersion` 5 → 6, lets transport self-reject stale IT callbacks by epoch).

---

2. AUDIO-RECOVERY-ROUTING-001 — timing-loss events reach the wrong backend (CONFIRMED)

Root cause

`IsochService` exposes a **single** `timingLossCallback_` slot. Both `DiceAudioBackend` and `AVCAudioBackend` wrap the same `IsochService` and both register into that slot from their constructors. `AudioCoordinator` constructs `dice_` before `avc_` (declaration order, `AudioCoordinator.hpp:74–75`), so **AV/C's registration deterministically overwrites DICE's** (last-writer-wins; introduced by PR #72 / FW-64, merge `e9ea6ce`).

Consequences

- An RX replay reset on a DICE device is delivered to `AVCAudioBackend::HandleTimingLoss`, whose `activeGuid_` never matches the DICE GUID → the event is **silently dropped** (`AVCAudioBackend.cpp:191–201`). DICE recovery never fires at all. This is a DICE stream-killer, independent of everything else in this report.
- Teardown UAF: the slot is an unsynchronized `std::function`, set/cleared on the service queue but invoked from the RX Poll (interrupt) context. `SetTimingLossCallback({})` during `BeginTeardown` races an in-flight invocation into a `[this]` capture of a backend that may be destructing.

Fix design

- `AudioCoordinator` owns the **single** subscription and routes by GUID via its existing `BackendForGuid()` (`AudioCoordinator.cpp:171–185`). Remove both backend-level registrations.
- Widen the event payload to a struct carrying `(guid, reason, rxEpoch, sessionEpoch)` — `rxReplayEpochResets` already exists on the RX side and is currently discarded.
- Serialize replace/clear/invoke: guard the slot (and `activeGuid_`) with a lock; snapshot under the lock, invoke outside it.
- Post-fix check: `grep -rn "SetTimingLossCallback" ASFWDriver/` must return only the coordinator registration plus the RX-level signal wiring.

Must land together with §1 — otherwise the recovery epoch can be minted for the wrong backend.

---

3. AVC-TX-EXPOSURE-001 — frames dropped when write frontier passes exposure horizon (CONFIRMED)

Root cause

`AmdtpPayloadWriter::WriteFloat32Interleaved` drops any host frame whose covering TX packet is not yet exposed (`absoluteFrame ≥ ExposedFrameEnd()` → `++withoutPacket; continue;` , `AmdtpPayloadWriter.cpp:100–111`) with **no retention**. The RX-derived TxAlign path later jumps the packetizer frame cursor forward (`ASFWAudioDriverZts.cpp:393–417` → `AlignFrameCursorOnce`), permanently abandoning the skipped interval. That is the audible click / silence, and — because future frames become writable again — the observed "self-heal without restart" while transport telemetry stays clean.

Cursor model (verified)

- **W** — CoreAudio write frontier (`sampleTime + ioBufferFrameSize`), advances on the RT thread every IO period (up to 512 frames with the active `DiceWorking1536` profile).
- **E** — `exposedFrameEnd_` , monotone, advanced only when the preparation queue runs `ExposeDataPacket` .
- **C** — `completionCursor` / `committedEnd` (DMA committed end).
- **R** — replay reader cursor (§4).

A ~9.5 ms preparation slip plus one 512-frame callback (~20 ms worst case) exceeds the incident's ~12 ms exposure lead → W crosses E. (Incident geometry figures 408/512/576 are pre-redesign; current code carries 912/1024/2400.)

Fix design (RT-safe pending-range retention)

- Retain the **absolute frame range** of unwritten frames — two plain `uint64_t` members private to the single RT writer (not a sample copy; the mapped host ring remains the storage of record).
- Lazy-flush at the top of the next `WriteFloat32Interleaved` against the now-larger E; bound work to O(frameCount).
- Declare a real xrun only when ring-wrap arithmetic proves the host ring overwrote the pending range (`frame < W − frameCapacity`) — count it, don't silently drop.
- Add an alignment-epoch atomic so TxAlign cannot silently abandon a pending interval; reset the pending range in `DiceTxStreamEngine::ResetForStart` and inside the §1 recovery epoch.

Orthogonal to §4 — does not preclude the replay-horizon redesign.

---

4. TX replay horizon — `RxSequenceReplay` structurally unfillable (CONFIRMED)

Root cause

The 678/912/1024 diagnosis in the document is verified correct:

- `PrepareTransmitSlots` (`ASFWAudioDriverZts.cpp:243`) calls `txReplayReader.TryRead` **exactly once per prepared TX packet**, with `maxToPrepare = kTxPreparationLeadPackets = 678` (caller `:641–642`).
- The reader `Begin()` s at `producer − 256` inside a **512-entry** ring (`RxSequenceReplay.hpp:138–139, 289–290`), so 1:1 consumption catches the producer at the 257th packet (`kAheadOfProducer` guard at `:317`).
- Since **678 > 512 = kCapacity**, the preparation lead is structurally unfillable from finite past RX — even starting at the oldest retained entry.
- Cold start compounds it: `ValidateTxPrefill` forces a full 912-NODATA prefill (~114 ms), past the 64 ms `overwritten` threshold.
- Constants confirmed: 678 = 144 + 534; 144 = 48 + 96; 534 = ceil((512+2400)·80/441) rounded to a 6-group; 912 = `kTxSharedSlotPackets` ; 1024 = `kTimelineSlots` ; `clientIoBudgetFrames` = 512.

Signature matches the incident exactly: `[TxPrepRange] short=0 marginAfter=678` , no `[RxReplayReset]` , repeated `[TxAlign]` .

Fix design (fitted SYT predictor)

Replace 1:1 ring consumption with a fitted `(offset, rate)` SYT predictor fed only by valid RX observations (`hasValidCip && syt != 0xffff && cadenceAccepted`); take DATA/NODATA from the deterministic AMDTP cadence; use the ring only as a bounded validation window. This removes the producer/reader cursor coupling entirely, so `ahead` / `overwritten` cannot occur by construction. Three-horizon separation maps to existing constants: full prefill = 912 (IT arm), PCM slot reservation = 534 (frame-exposure window, needs zero RX timing), SYT prediction = predictor with validity horizon H. Predictor epoch must equal the §1 session epoch.

Drift budget verified: ~208 ticks accumulate over the 84.75 ms lead at 100 ppm — less than half a sample (256 ticks) — so a single fit extrapolates cleanly across the lead.

> **Correction to the document:** the hard staleness horizon at 100 ppm is **~0.104 s**, not 0.417 s (a 4× arithmetic slip). No conclusion changes; any such H comfortably exceeds the 84.75 ms lead.

Telemetry note: the requested first-fault telemetry largely exists as `[TxReplay] fail=…` (`ASFWAudioDriverZts.cpp:259–275`), but it is rate-limited (can miss the first fault of an epoch) and the non-fatal NODATA branch lacks the four discriminating fields (readerNext, producer, epoch, packet index). A once-per-epoch latch fixes both.

---

5. Original RX timing loss (AVC-HEALTH-001) — new concrete defect found

This finding is **new** — it appears in neither the document nor the third-party report, and it is the strongest candidate for the *original* trigger.

What is ruled out

- The RX timestamp/ordinal arithmetic is **sound**: the 128-second seconds-field wrap and the 1,024,000 cycle-ordinal wrap reconstruct correctly; each packet's timestamp is reconstructed independently (no chained accumulator); cursors re-seed on every reset. There is **no accumulating-drift path**, so "fails after minutes with no bus reset" is not arithmetic.
- The IR path is packet-per-buffer (504 × 4096 B), so it has **no analogue** of the AR 4160-byte straddler bug (the CoolScan wedge class).
- `clock-anchor-rejected` is effectively unreachable (the caller pre-checks both rejection conditions).

The defect

`DirectAudioReceiveConsumer::ConsumePacket` early-returns on `packet.payload.empty()` (`DirectAudioReceiveConsumer.cpp:150`) **before** advancing the cycle cursor. An errored/zero-length IR descriptor (`xferStatus != 0`, `resCount == reqCount`) is drained by the ring but skips `lastReplayCycleOrdinal_`, so the next good packet reads a `receive-cycle-gap` of **exactly +1** — a single lost packet mis-attributed as a timing gap and escalated (via §2's routing and §1's broken recovery) into a fatal stream loss.

Discriminating next step

Capture the first `[RxReplayReset]` record on hardware: the reason string selects the failing layer, and within `receive-cycle-gap` the sign and magnitude of `(observed - expected)` separates drop-of-1 (this defect) from ring overrun and duplicate re-drain. Fix direction: tolerate an isolated cycle discontinuity (advance and count) instead of resetting the established replay epoch on the first one. Additionally, decode the OHCI event code (`xferStatus[4:0]`) in the IR drain — the async `RxPath` already does; the isoch drain does not — so an errored completion is classified rather than silently skipped. Cross-check the zero-byte-error-completion behavior against Linux `drivers/firewire/ohci.c` before finalizing (see Limitations).

---

6. Verified-sound / already-fixed items

- **AVC-SYNC-001 (raw RX/TX SYT mismatch as primary cause): not supported.** The SYT doctrine in current code is compliant: no cross-direction raw-SYT comparison, `0xffff` never feeds cadence, empties keyed on `hasValidCip` + SYT (not FDF), full-cycle expansion helpers present and used, RX/TX transfer delays kept separate. Treat "raw SYT inequality" as a false lead.
- **AVC-PROPERTY-001: fix already landed.** The nub publishes `ASFWStreamMode` and the parser reads it (`Audio/DriverKit/Config/AudioDriverConfig.cpp:78-83`). Confirm on the next Duet/BeBoB start via the log line `ASFWAudioDriver: Stream mode from nub: blocking`.

- **AVC-RECOVERY-002 (escalation policy): confirmed half-running state.** The destructive escalation (transport-only restart, HAL never informed) should be gated until §1 lands. Interim: delete the escalation branch at `AVCAudioBackend.cpp:249–288` — the **whole** branch including the attempt-budget counting at `:251–264` (deleting only `:266–288` orphans the counter) — keep the settle window + self-heal detection (`IsReceiveReplayEstablished`), emit `[Recovery]` telemetry. Next increment: request a HAL-bracketed restart via `RequestDeviceConfigurationChange` so StopIO/StartIO re-run the producer protocol. None of the five reset reasons alone justifies an automatic destructive restart; only corroborated outages (bus reset, unplug, clock-source change) do.

Recommended landing order

1. **§1 + §2 together** — the producer-owned recovery epoch and the coordinator-owned GUID router share the epoch/session concept and must not be split (the routing fix must not re-enable an unsafe DICE-path escalation).
2. **Gate the escalation (§6, AVC-RECOVERY-002 interim)** — cheap safety so a still-broken recovery can't leave a half-running CMP.
3. **Empty-payload cycle-accounting fix + first-fault telemetry latch (§5)** — small, and it stops mis-attributing transients as the fault that triggers everything above.
4. **Replay predictor (§4) and pending-range guard (§3)** — larger, orthogonal, defense-in-depth; land after the recovery path is safe.

Proposed regression tests

Test	Type	Pass condition
Timing-loss router dispatches by GUID+epoch	unit	DICE and AVC each receive only their own events; no overwrite
No auto-restart without producer epoch	unit/ integration	timing loss logs/xruns but never invokes duplex restart without a prepared epoch
Recovery epoch resets producer before prime	integration	recovered start always sees `48 ≤ committedEnd ≤ 912` and validated prefill
PayloadWriter retains ahead-of-exposure frames	unit	frames beyond exposure become pending, not `framesWithoutPacket` loss
RxReplayReset first-fault reason matrix	unit	each reset-reason class emits one correct first-fault record per epoch
Empty-payload packet advances cycle cursor	unit	zero-length drained descriptor does not produce a `receive-cycle-gap +1` on the next packet

Limitations

- **Reference stacks not source-verified in this checkout.** `references/` (gitignored) currently contains only `IOFireWireSerialBusProtocolTransport` ; the Linux OHCI/ALSA stacks,

`IOFireWireFamily.kmodproj`, and libffado are absent. All Apple/FFADO/Linux behavioral citations (recovery ordering, `amdtp-stream.c` SYT generation, FFADO fitted-rate validation, `ohci.c` IR event semantics) come from the document's own notes, not independent source reads. Per the wire-compat doctrine, clone those before finalizing any wire-observable change — in particular the §5 zero-byte IR error-completion behavior needs `drivers/firewire/ohci.c` confirmation.

- The incident value `committedEnd ≈ 13,626,402` is illustrative (no runtime capture in-repo) but consistent with the confirmed Prime-rejection logic.
- No hardware run was performed for this triage; §5's discriminating telemetry defines the single HW capture needed next.